

Семинарно упражнение 1: Въведение

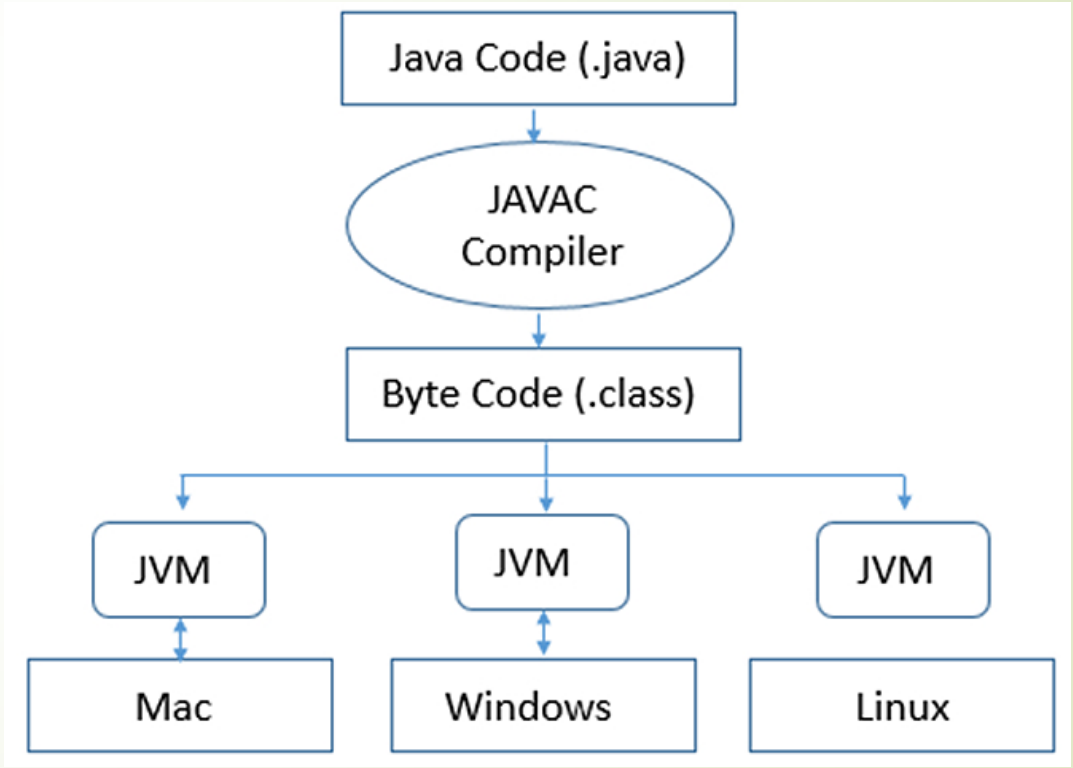
Инж. Лилия Андонова
ТУ – София
ФКСТ

Упражнението днес:

- Виртуална машина.
- Развойни среди.
- Създаване на проект.
- Структура на проект.
- Създаване на пакет и „Main“ клас.
- Вход и изход в конзолата.
- Аритметични оператори
- Логически оператори
- If-else оператори
- Swich оператори
- For цикъл
- While цикъл
- Do-while цикъл
- Клас „Arrays“.
- Вградени методи за работа с масиви.
- Подаване на масив като параметър на метод.
- Отпечатване, сравнение и сортиране на масиви.
- Търсене в масив.
- Какво е метод и видове методи

Java Virtual Machine

Java Virtual Machine (или JVM) е абстрактна изчислителна машина, която е основният компонент, осигуряващ независимостта от хардуер и операционна система.



Java виртуалната машина (JVM)

- Интерпретира и изпълнява byte код инструкции
- Компилира по време на изпълнението байт кода до машинен код
- Заделя памет за оперативните данни
- Автоматично изчиства паметта
- Зарежда класове
- Стартира нишки
- Взаимодейства с операционната система

Развойни среди (IDE)

Съвкупност от инструментални програми, необходими за създаване на нови програми се нарича система за програмиране.

Елементите на системата за програмиране са текстов редактор, компилатор, свързващ редактор, интерпретатор, дебъгер, библиотека от подпрограми.

Интегрирането на всички елементи на системата за програмиране в едно цяло се нарича интегрирана среда за програмиране ***(Integrated Development Environment)*.**

Основни елементи на IDE

Текстов редактор - кодът се пише във файла като се използва текстов редактор;

Транслатор – превежда кода на машинен език;

Дебъгер – позволява програмата да се изпълни стъпка по стъпка, като на всяка стъпка се вижда какво действие се изпълнява и се отстраняват логическите грешки (бъгове).

IDEs за JAVA

Eclipse

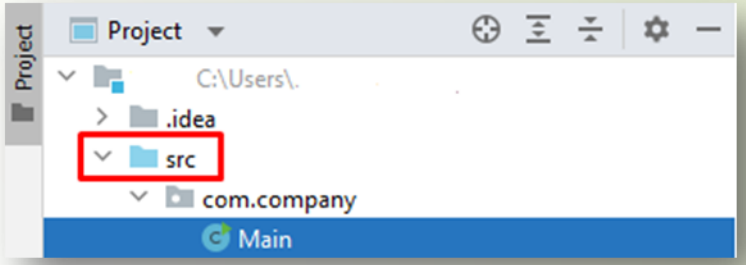
NetBeans

IntelliJ IDEA - един от най-предпочитаните инструменти от професионалните Java разработчици.

- Разработен от JetBrains;
- Интелигентно допълване на код;
- Вграден анализ на кода и автоматични предложения за подобрение.

Структура на проект

Програмният код се съхранява в папка src на проекта



Сорс кодът на програма се пише в секцията main(String[] args)

Между отварящата и затварящата скоба { }

Кодът на програмата се пише отместен навътре

```
package com.company;

public class Main {
    public static void main(String[] args) {
    // write your code here
    }
}
```

Пакети (Packages)

Пакетите са логически групи от класове.

В src/main/java се създават папки, които представят йерархията на пакетите.

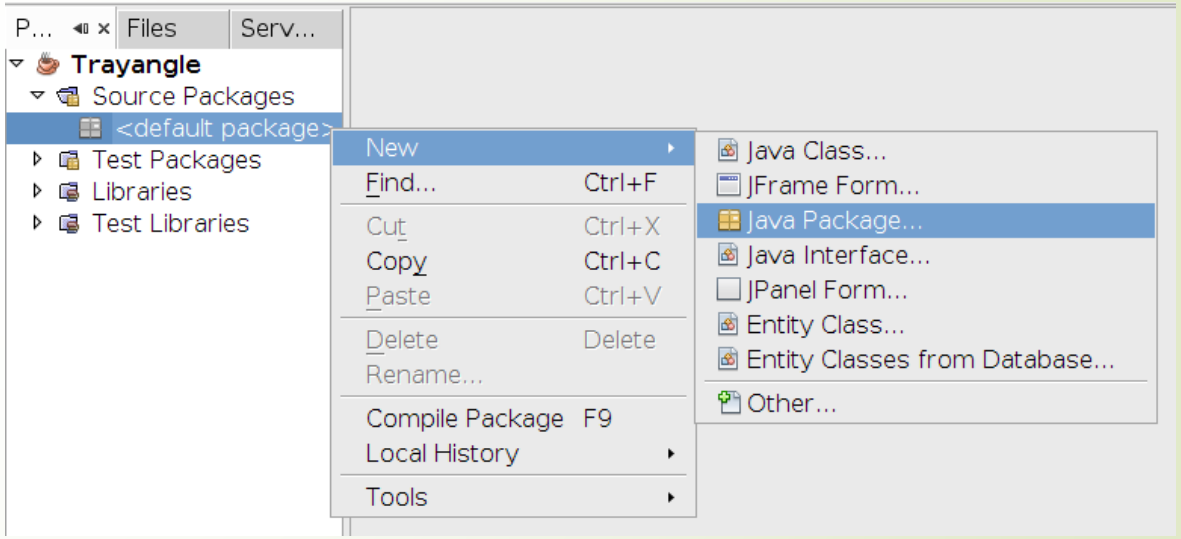
Един IntelliJ IDEA проект съдържа един или повече Java сорс файла, конфигурационни файлове и други ресурси.

Във всеки Java сорс файл има една или повече дефиниции на типове (класове или други дефиниции).

В класовете има методи (действия), а те се състоят от поредици от команди.

Създаване на пакет

В папка src с десен бутон избираме Package и създаваме папка.



Клавишни комбинации в IntelliJ IDEA

- [CTRL + F] Комбинацията отваря търсачка, с която можем да търсим в нашия код.
- [CTRL + /] Закоментира част от кода. / Разкоментира код, който е вече закоментиран.
- [CTRL + Z] Връща една промяна назад (т.нар. Undo).
- [Ctrl + Shift + Z] Комбинацията има противоположно действие на [CTRL + Z] (т.нар. Redo).
- [CTRL + ALT + L] Форматира кода според конвенциите по подразбиране.
- [CTRL + Backspace] Изтрива думата вляво от курсора.
- [CTRL + Del] Изтрива думата вдясно от курсора.
- [CTRL + S] Запазва всички файлове в проекта.
- [CTRL + D] Копира ред или маркирания фрагмент.
- [CTRL + Y] Изтрива текущия ред.

Именуване на проекти, пакети и класове

За именуване на **проекти и класове** се препоръчва описателно име, което подсказва каква е ролята на въпросния клас/ проект и в същото време се препоръчва **PascalCase** конвенцията. Това е конвенция за именуване на елементи, при която всяка дума, включително първата, започва с главна буква, например ExpressionCalculator.

„Main“ клас

Сорс кодът на Java програмите се пише в секцията **main(String[] args)**, между отварящата и затварящата скоба { }.

Това е главният метод (действие), което се изпълнява при стартиране на една Java програма.

Натискаме [Enter] след отварящата скоба { и започваме да пишем.

Кодът на програмата се пише отместен навътре, като това е част от оформянето на текста, за по-голямо удобство при повторен преглед и/или дебъгване.

public static void main(String[] args) {.....}

public: методът е достъпен от всяко място, дори извън класа, в който се намира.

static: методът може да бъде извикан без да се създава инстанция на класа. Понеже JVM трябва да стартира приложението без да създава инстанция на класа, методът трябва да бъде статичен.

void: Методът не връща никаква стойност.

main: Това е името на метода, който JVM търси, за да стартира Java програмата.

String[] args: Това е параметърът, който представлява масив от аргументи, подадени на програмата от командния ред.

Системна конзола

- обикновено наричана само «конзола»
- устройството, чрез което подаваме команди на компютъра в текстов вид и получаваме резултатите от тяхното изпълнение отново като текст.
- в повечето случаи представлява текстов терминал, т.е. приема и визуализира само текст, без графични елементи като например бутони, менюта и т.н.
- изглежда като прозорец с черен цвят, в който пишем и който отпечатва текстов резултат
- в конзолата се изпълняват конзолни приложения
- чете текстов вход и печата изхода като текст

Променливи

- Компютрите са машини, които обработват данни
- Данните се записват в компютърната памет в променливи
- Променливите имат три основни характеристики: тип, име и стойност
- Променливите трябва да се дефинират преди употреба
- Променливите могат да се дефинират навсякъде в блока
- Инициализацията на променливите е задължителна
- Дефиниране на променлива и присвояване на стойност:

  <тип> <име> = <стойност>;

Типове данни

- Типът задава информация за допустимите стойности на една променлива.
- Типовете се ползват от компилатора за проверка за типова съвместимост в програмата.
- Дефинират и размера на паметта, използван за съхранение на данните.
- Java има вградени базови типове:
  - boolean - true или false
  - byte,short,int,long - 1,2,4,8 byte цели числа
  - float,double - 4 и 8 byte числа с ПЗ
  - char – символен тип
  - void - ползва се при функции, които не връщат стойност
- Java поддържа и масиви.
- Java поддържа и класове.

Четене от конзолата

- Всичко, което получаваме от конзолата, идва под формата на текст
- Всичко, което печатаме на конзолата, се преобразува в текст

В Java има няколко начина да прочетем някакъв вход от конзолата. Единият е чрез използването на вградения клас Scanner.

За да прочетем данни от конзолата е необходимо първо да импортираме класа Scanner на първия ред от нашата програма, още преди декларацията на основния клас. Слагаме този ред най-отгоре:

import java.util.Scanner;

След това в main(…) метода на програмата трябва да създадем нова **Scanner инстанция**, която да закачим за системния вход.

Scanner scanner = new Scanner(System.in);

След това през създадения сканиращ четец scanner можем да четем цели и дробни числа, текст и други данни.

Команда за четене от конзолата

Връща ни текста, въведен от потребителя

```
Scanner scanner = new Scanner(System.in);
String name = scanner.nextLine();
System.out.println(name);
```

Четене на цели числа от конзолата

```
Scanner scanner = new Scanner(System.in);
String input = scanner.nextLine();
int num = Integer.parseInt(input);
```

```
Scanner scanner = new Scanner(System.in);
int a = Integer.parseInt(scanner.nextLine());
```

/Подходящо е, когато четем ред по ред (например ако потребителят въвежда стойности на отделни редове)/

```
Scanner scanner = new Scanner(System.in);
int num = scanner.nextInt();
```

- */Подходящо е, когато въвеждаме числа, разделени с интервал,* табулация *или нов редове/*

Четене на дробни (реални) числа

```
Scanner scanner = new Scanner(System.in);
String input = scanner.nextLine();
double num = Double.parseDouble(input);
```

```
Scanner scanner = new Scanner(System.in);
double num = Double.parseDouble(scanner.nextLine());
```

```
Scanner scanner = new Scanner(System.in);
double num = scanner.nextDouble();
```

Отпечатване на конзолата

Отпечатване на текст:

System.out.print(„a= ");

Отпечатване на стойност на променлива:

System.out.print**ln**(a);

Съединяване на текст и числа

При печат на текст, числа и други данни, можем да ги съединим, използвайки шаблони

%s (String), %d (int), %f (double), %c (char), %n (new line),…

```
Scanner scanner = new Scanner(System.in);
String firstName = scanner.nextLine();
String lastName = scanner.nextLine();
int age = Integer.parseInt(scanner.nextLine());
String town = scanner.nextLine();
System.out.printf("You are %s %s, a %d-years old person from %s.", firstName, lastName, age, town);
```

Оператори

- Във всички езици за програмиране се използват оператори, чрез които се изразяват някакви действия върху данни.
- Операторите позволят манипулиране на примитивни типове данни.
- Те са символи, които извършат специфични операции над един, два или три операнда и връщат резултат от извършените операции.
- Операторите в Java могат да бъдат разделени в няколко различни категории:

  Аритметични – също както в математиката, служат за извършване на прости математически операции.

  Оператори за присвояване – позволяват присвояването на стойност на променливите.

  Оператори за сравнение – дават възможност за сравнение на два литерала и/или променливи.

  Логически оператори – оператори за работа с логически типове данни.

  Побитови оператори – използват се за извършване на операции върху двоичното представяне на числови данни.

  Оператори за преобразуване на типовете – позволяват преобразуването на данни от един тип в друг.

Унарни оператори

- ***Унарните оператори са тези оператори в Java, които се нуждаят само от един операнд, за да изпълнят някаква функция.***
- ***Те работят на същия принцип като унарните операции в математиката.***
- ***Например, можете да използвате унарни оператори, за да представите положителна или отрицателна стойност, да увеличите или намалите стойност с 1 или да отхвърлите стойност.***

оператор	действие
++	инкрементиране с 1;
– –	декрементиране с 1;
+	положително число;
–	отрицателно число;
~	побитово инвертиране;
!	логическо отрицание;
(<тип>)	преобразуване на тип данни

инкрементиране ++
декрементиране --

Операторът ++ увеличава числото с единица (a = a + 1;)

- постфикс a++ (първо изпълнява израза и след това увеличава с 1)
- суфикс ++a (първо увеличава с 1 и след това извършва действията)

Операторът -- намалява числото с единица (a = a – 1;)

- постфикс a-- първо изпълнява израза и след това намалява с 1)
- суфикс --a (първо унамалява с 1 и след това извършва действията)

Аритметични оператори

Аритметичните оператори служат за извършване на математически операции между числа.

оператор	действие
+	събиране
-	изваждане
*	умножение
/	деление
%	модулно деление

Аритметични операции: + и -

- Събиране на числа (оператор +):

```
int a = 10;
int b = 3;
System.out.println("Събиране: " + (a + b)); // 13
```

- Изваждане на числа (оператор -):

```
int a = 10;
int b = 3;
System.out.println(„Изваждане: " + (a - b)); // 7
```

Аритметични операции: * и /

- Умножение на числа (оператор *):

```
int a = 10;
int b = 3;
System.out.println(„Произведение: " + (a * b)); // 130
```

- Деление на числа (оператор /):

```
int a = 10;
int b = 3;
System.out.println(„Частно: " + (a / b)); // 3 – дробната част се отрязва
double c = a / 3.0; // 3.3333333333 – дробно деление
System.out.println(„Частно: " + (a / 0)); // Грешка: деление на 0
```

Аритметични операции: %%

- Модул - остатък от целочислено деление на числа (оператор %):

```
int a = 7;
int b = 2;
int product = a % b;  // 1


int odd = 3 % 2; // 1 – числото е нечетно
int even = 4 % 2; // 0 – числото е четно
int error = 3 % 0; // Грешка: деление на 0
```

Операторите за сравнение

- използват се за сравняване на две или повече операнди.
- върнатият от тях резултат е булев (true или false)
- имат по-малък приоритет от аритметичните

оператор	действие
>	по-голямо
<	по-малко
>=	по-голямо или равно
<=	по-малко или равно
==	оператора за равенство
!=	различие

Логическите оператори

- Логическите оператори работят с булеви стойности (true и false) и служат за комбиниране на условия.
- Основни логически оператори:

&& (логическо „И“) – връща true, ако и двете условия са верни.

|| (логическо „ИЛИ“) – връща true, ако поне едно от условията е вярно.

! (логическо „НЕ“) – променя стойността на булевата променлива – от true на false и обратно.

^ (изключващо ИЛИ) – връща резултат true, когато само един от двата операнда има стойност true.

Условни оператори в Java

- използват се за вземане на решения въз основа на резултата от дадено условие

- позволяват на програмата да изпълнява различни действия в зависимост от това дали условието е вярно или невярно.

Оператор if-else

При false на условието, можем да изпълним други действия - чрез else конструкция

```
if (условие) {
    // код, който се изпълнява, ако условието е вярно
} else {
    // код, който се изпълнява, ако условието е невярно
}
```

Вложени if конструкции

- Влагане наричаме поставянето на if или if-else клауза в друга if или else конструкция.
- Всяка else клауза се отнася за най-близко разположената предходна if клауза.
- Не е добра практика нивото на влагане да бъде повече от три.
- Използвайте блокове, заградени с къдрави скоби { } с цел избягване на двусмислие;
- Използвайте else клауза след всяка if клауза, когато е възможно;
- Поставяйте условните ситуации, които желаете да бъдат изпълнени първи на първо място;
- Форматирайте винаги програмния код, с цел да бъде лесно четим и да не позволява двусмислие;

Серия от проверки if-else if-else

Когато има нужда от проверка на повече от едно условие, може да използваме няколко if-else if-else структури.

```
if (условие1) {
    // код за условие1
} else if (условие2) {
    // код за условие2
} else {
    // код, ако нито едно от условията не е изпълнено
}
```

Тази структура позволява проверка на няколко различни стойности.

Оператор switch

- Операторът switch се използва, когато трябва да се проверяват множество стойности на една и съща променлива.
- Той е по-ефективен от многожество последователни if-else конструкции.

```
switch (променлива) {
    case стойност1:
        // код за стойност1
        break;
    case стойност2:
        // код за стойност2
        break;
    default:
        // код в случай на липса на съвпадение при case
}
```

Добри практики при използване на switch-case

- Добра практика при използването на конструкцията за избор на вариант switch е default конструкцията да бъде поставяна на последно място, с цел програмния код да бъде по-лесно четим.
- Важно е да се използват отделни case конструкции за обработка на различни ситуации.
- Добре е на първо място да бъдат поставяни онези case конструкции, които обработват най-често възникващите ситуации. case конструкциите, които обработват ситуации, възникващи по-рядко могат да бъдат поставени на последно място.
- Добре е case конструкциите да бъдат подреждани в азбучен ред, ако целочислените стойности, с които се сравнява селекторът, са от символен тип.
- Добре е case конструкциите да бъдат подреждани в нарастващ ред, ако целочислените стойности, с които се сравнява селекторът, са от целочислен тип.
- Добре е да се използва default конструкция за ситуации, които не могат да бъдат обработени при нормално изпълнение на програмата.

Цикли

- Циклите се използват за повторно изпълнение на блок от код определен брой пъти или докато условие е вярно.
- Цикълът (loop) е структурата, която позволява това изпълнение без излишно писане на повтарящ се код.
- Цикъл, който никога не свършва, се нарича безкраен (infinite).

For цикъл

- всяко повторение се нарича "итерация;
- броячът може да е са нарастваща или увеличаваща стъпка;
- стъпката на брояча може да е различна от 1;
- for циклите не работят само с числа
- **break**: прекъсва изпълнението на цикъла и излиза от него веднага, дори ако условието все още е вярно;
- **continue**: пропуска текущата итерация и преминава към следващата, без да прекъсва целия цикъл

For цикъл

Цикълът for се използва, когато знаем предварително колко пъти трябва да се изпълни даден блок от код.

Синтаксис:

```
for (инициализация; условие; инкремент/декремент) {
    // код, който се изпълнява многократно
}
```

Пример:

```
for (int i = 0; i < 5; i++) {
    System.out.println("i = " + i);
}
```

- Инициализация – задаваме началната стойност на брояча. Тази част се изпълнява само веднъж, в началото на цикъла.
- Условие – логически израз, който се проверява преди всяка итерация. Ако условието е вярно (true), тялото на цикъла се изпълнява. Ако е невярно (false), цикълът приключва.
- Инкремент/Декремент – изпълнява се в края на всяка итерация и обикновено променя стойността на брояча, за да приближи цикъла към края му.

While цикли

Цикълът while продължава да изпълнява даден блок от код, докато условието е вярно.

```
while (условие) {
    // код, който се изпълнява многократно
}
```

Условието трябва да дава boolean резултат.

Семантика:

- ✓ изчислява се условието
- ✓ false – изход от цикъла
- ✓ true – изпълнява се тялото на цикъла

Ако условието на while цикъла никога не стане невярно, то цикълът ще продължи да се изпълнява безкрайно, което води до безкраен цикъл.

Do-while цикъл

Цикълът do-while е подобен на while, но първо изпълнява блока с код поне веднъж и след това проверява условието.

Синтаксис:

```
do {
    // код
} while (условие);
```

Тяло на цикъла: блокът от код вътре в do се изпълнява първоначално, дори ако условието е невярно.

Условие: логически израз, който се проверява след всяка итерация. Ако условието е вярно, цикълът продължава да се изпълнява. Ако е невярно, цикълът приключва.

Ако условието на do-while цикъла никога не стане невярно, това води до безкраен цикъл.

Какво е масив?

- Масивът (array) е структура от данни, която съхранява фиксиран брой елементи от един и същ тип.
- Масивът е референтен тип.
- Всеки елемент в масива се достъпва чрез индекс, който започва от 0 и завършва дължината – 1 (numbers.length)
- Масивите са удобни за съхраняване на списъци от елементи, като числа, символи или обекти.



Създаване на масив

- Деклариране на масив
- Дефиниране на масив
- Инициализиране на масив

- тип[] имеНаМасива;
- тип[] имеНаМасива = new тип[размер];
- int[] numbers = {1, 2, 3, 4, 5};

Достъп и промяна на елемент

- **Достъп до елементите на масива**

  Можем да достъпваме елементите в масива чрез индексиране. Индексите започват от 0 и стигат до размер - 1.

  int[] numbers = {10, 20, 30, 40, 50};

  System.out.println(numbers[0]); // Извежда 10

  System.out.println(numbers[4]); // Извежда 50

- **Промяна на елемент в масива**

  Можем да променяме стойностите в масива, като зададем нова стойност за даден индекс.

  numbers[1] = 25; // Задаваме нова стойност на втория елемент

Двумерни масиви (Матрици)

- Java поддържа ***многомерни масиви***. Най-често използваният многомерен масив е ***двумерен масив (матрица)***, който може да се представи като таблица с редове и колони.
- **Създаване на двумерен масив:**

```
int[][] matrix = new int[3][3]; // Матрица с 3 реда и 3 колони
```

- **Инициализиране на двумерен масив:**

```
int[][] matrix = {    {1, 2, 3},    {4, 5, 6},    {7, 8, 9}};
```

- **Достъп до елемент в двумерен масив:**

```
System.out.println(matrix[1][2]); // Достъпва елемента на втория ред и третата колона (стойност 6)
```

- **Обхождане на двуизмерен масив:**

```
for (int i = 0; i < matrix.length; i++) {
for (int j = 0; j < matrix[i].length; j++)
{       System.out.print(matrix[i][j] + " ");
}   System.out.println();}
```

Класът Arrays в Java

- Класът Arrays е част от пакета java.util.
- Съдържа статични методи за манипулиране с масиви.
- Тези методи са удобни за работа с масиви, като включват отпечатване на съдържанието на масиви, сортиране, сравняване, търсене на елементи и други полезни операции.
- import java.util.Arrays;

Методи (1)

- **Arrays.sort()** – Сортиране на масив. Методът sort() сортира елементите в масива в нарастващ ред.

```
int[] numbers = {5, 3, 8, 1, 2};
Arrays.sort(numbers);
System.out.println(Arrays.toString(numbers)); // Изход: [1, 2, 3, 5, 8]
```

- **Arrays.binarySearch()** – Бинарно търсене. Методът binarySearch() се използва за намиране на индекс на елемент в сортиран масив. Връща индекса на намерения елемент или отрицателна стойност, ако елементът не съществува в масива.

```
int[] numbers = {1, 2, 3, 5, 8};
int index = Arrays.binarySearch(numbers, 3);
System.out.println("Index of 3: " + index); // Изход: Index of 3: 2
```

- **Arrays.fill()** – Запълване на масив. Методът fill() позволява да запълним всички елементи на масив с една и съща стойност.

```
int[] numbers = new int[5];
Arrays.fill(numbers, 7);
System.out.println(Arrays.toString(numbers)); // Изход: [7, 7, 7, 7, 7]
```

Методи (2)

- **Arrays.equals()** – Сравняване на масиви. Методът equals() сравнява два масива и връща true, ако те съдържат същите елементи в същия ред.

```
int[] array1 = {1, 2, 3};
int[] array2 = {1, 2, 3};
System.out.println(Arrays.equals(array1, array2)); // Изход: true
```

- **Arrays.copyOf()** – Копиране на масив. Методът copyOf() създава копие на масива с определена дължина. Ако новата дължина е по-голяма от оригиналния масив, новите елементи ще бъдат инициализирани със стойност по подразбиране.

```
int[] original = {1, 2, 3};
int[] copy = Arrays.copyOf(original, 5);
System.out.println(Arrays.toString(copy)); // Изход: [1, 2, 3, 0, 0]
```

- **Arrays.copyOfRange()** – Копиране на част от масив. Методът copyOfRange() копира част от масива, започвайки от даден начален индекс до краен индекс.

```
int[] numbers = {10, 20, 30, 40, 50};
int[] partialCopy = Arrays.copyOfRange(numbers, 1, 4);
System.out.println(Arrays.toString(partialCopy)); // Изход: [20, 30, 40]
```

Методи (3)

- **Arrays.toString()** – Преобразуване на масив в низ. Методът toString() преобразува масив в низ, което е полезно за извеждането му на конзолата.

```
int[] numbers = {1, 2, 3, 4, 5};
System.out.println(Arrays.toString(numbers)); // Изход: [1, 2, 3, 4, 5]
```

- **Arrays.asList()** – Преобразуване на масив в списък. Методът asList() създава фиксиран списък от елементите на даден масив. Полезно е, когато искаме да използваме методите на List върху масив.

```
String[] fruits = {"Apple", "Banana", "Orange"};
List<String> fruitList = Arrays.asList(fruits);
System.out.println(fruitList); // Изход: [Apple, Banana, Orange]
```

- **Arrays.setAll()** – Промяна на всички елементи в масива. Методът setAll() позволява да зададете стойност за всички елементи в масива чрез ламбда израз или функция. int[] numbers = new int[5];

```
Arrays.setAll(numbers, i -> i * 2); // Задава на всеки елемент стойност равна на индекса му * 2
System.out.println(Arrays.toString(numbers)); // Изход: [0, 2, 4, 6, 8]
```

Подаване на масив като параметър на метод

В Java можем да подаваме масиви като параметри на методи, както всеки друг тип данни.

Масивът се предава по референция, което означава, че всяка промяна, направена върху масива вътре в метода, ще бъде отразена и в оригиналния масив.

```
public class Example {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        printArray(numbers);
    }
    public static void printArray(int[] array) {
        for (int i : array) {
            System.out.print(i + " ");
        }
  }
}
```

Обхождане на масив

Обхождането на масив най-често се прави с цикъл **for** или **for-each**, за да се премине през всички елементи.

- ```
  int[] numbers = {10, 20, 30, 40, 50};
  for (int i = 0; i < numbers.length; i++) {
  System.out.println("Element at index " + i + ": " + numbers[i]);
  }
  ```
- ```
  for (int num : numbers) {
  System.out.println(num);
  }
  ```

Отпечатване на масиви

За да отпечатаме масив в четим вид, можем да използваме метода Arrays.toString(). Този метод преобразува масива в низ (String), който може да бъде отпечатан с помощта на System.out.println().

```
int[] numbers = {1, 2, 3, 4, 5};

System.out.println(Arrays.toString(numbers));// [1, 2, 3, 4, 5]
```

Сравнение на масиви

Методът Arrays.equals() позволява да се сравнят два масива елемент по елемент. Той връща true, ако двата масива имат еднакви дължина и съдържание.

```
int[] array1 = {1, 2, 3};
int[] array2 = {1, 2, 3};
int[] array3 = {4, 5, 6};

System.out.println(Arrays.equals(array1, array2)); // true
System.out.println(Arrays.equals(array1, array3)); // false
```

Сортиране на масиви

За сортиране на масиви в Java можем да използваме метода Arrays.sort(). Той сортира масива по възходящ ред (от най-малкия към най-големия елемент). Ако масивът съдържа обекти (напр. низове), те се сортират по азбучен ред.

```
int[] numbers = {5, 1, 4, 2, 3};

Arrays.sort(numbers);

System.out.println(Arrays.toString(numbers));// [1, 2, 3, 4, 5]
```

Какво е метод?

- Именуван блок от код, който може в определен момент да бъде извикан
- Променливите, декларирани в метода, са локални
- Пример за **дефиниция на метод**:
  - Декларира се навсякъде между отварящата и затварящата скоба на дадения клас, но не и в рамките на друг метод или конструктор.
  - Общата структура е:

[модификатори] [тип_връщан_резултат] имеМетод(параметри) {

тяло на метода – инструкции

}

  - Модификатори (по желание)

    public, private, protected → определят достъпа.

    static → методът принадлежи на класа, а не на обект.

    final, abstract → специални случаи.
  - Тип на връщан резултат – типът на стойността, която методът връща. Ако не връща нищо пишем void.
  - Име на метода – следва конвенция: започва с малка буква, описва действието.
  - Параметри (в скоби) – списък от входни данни за метода. Ако няма оставяме празни скоби ().
  - Тяло на метода (в { }) – съдържа кода, който ще се изпълни при извикване.

- **Сигнатура на метод:** съвкупност от атрибути, които идентифицират еднозначно един метод. Сигнатура или спецификация на метода се състои от двойката име и списък с параметри.
- **Имплементация** – представлява тялото на метода, заключено между отваряща и затваряща къдрава скоба след декларацията на метода. Логиката и функционалността на метода се намират именно в тялото на метода.
- **Локални променливи** – всички променливи, които са декларирани в рамките на даден метод. Те са достъпни само и единствено в метода. Тоест областта им на съществуване е заключена между реда на декларацията и затварящата скоба на метода.
- **Блок** – наричаме програмен фрагмент заключен между две къдрави скоби. Ако декларираме променлива в блок, то тя отново е локална и важи горното правило за областта и на съществуване. Тялото на метода представлява програмен блок.

Какво е void метод?

Метод, който не връща стойност.

- Използва се, когато искаме да извършим действие (например да отпечатаме текст, да променим данни, да запишем файл), но няма нужда да връщаме резултат.
- Синтаксис:

  [модификатор] void имеМетод(параметри) {

  ……

  }
- Основни характеристики
  - Вместо тип на връщаната стойност пишем void.
  - Не можем да използваме return със стойност (но можем просто return;, за да прекъснем метода).
  - Извикването на void метод е самостоятелна инструкция – не може да се използва в израз, защото не връща резултат.

Методи с връщана стойност

- Метод с връщана стойност е такъв метод, който изчислява резултат и го връща на мястото, където е извикан.
- Синтаксис:

```
[модификатор] [тип] имеМетод(параметри) {
   ……….
   return стойност;
}
```

- Основни характеристики
  - Типът на връщаната стойност – винаги се посочва (например int, double, String и т.н.).
  - Операторът return е задължителен. Той връща резултата и прекратява метода.
  - Стойността след return трябва да е същия тип като посочения.

Защо са полезни методите с връщана стойност?

- Позволяват преизползване на резултата (например можеш да го запазиш в променлива или да го подадеш като аргумент в друг метод).
- Правят кода по-структуриран и четим.
- Дават възможност за по-лесно тестване – може да провериш какво връща даден метод.
- Може да има само един тип връщане, но няколко return в различни клонове:

```
public int abs(int x) {
        if (x >= 0) {
        return x;
        } else {
         return -x;
       }
       }
```

- Винаги трябва да има return за всички пътища на изпълнение, иначе компилаторът дава грешка.

Извикване на метод

- Извикването е мястото в програмата, където реално използваме метода.
- Когато извикаме метод:
  - Управлението на програмата „прескача“ в метода.
  - Изпълняват се инструкциите му.
  - Ако има return, стойността се връща на мястото на извикването.
  - Програмата продължава след извикването.
- Общ синтаксис за извикване

  имеНаМетод(аргументи);
- Ако методът е void, извикването е самостоятелна инструкция.
- Ако методът връща стойност, извикването може да е:
  - в израз
  - в присвояване
  - директно в System.out.println()

Какво научих днес:

- ✓ Виртуална машина.
- ✓ Развойни среди.
- ✓ Създаване на проект.
- ✓ Структура на проект.
- ✓ Създаване на пакет и „Main“ клас.
- ✓ Вход и изход в конзолата.
- ✓ Аритметични оператори
- ✓ Логически оператори
- ✓ If-else оператори
- ✓ Swich оператори
- ✓ For цикъл
- ✓ While цикъл
- ✓ Do-while цикъл
- ✓ Клас „Arrays“.
- ✓ Вградени методи за работа с масиви.
- ✓ Подаване на масив като параметър на метод.
- ✓ Отпечатване, сравнение и сортиране на масиви.
- ✓ Търсене в масив.
- ✓ Какво е метод и видове методи